

Technical Requirements Document (TRD)

Suraksha Shadow – Women Safety & Social Impact

Version: 1.0 · **Companion to:** Suraksha Shadow FRD

1. Purpose

This document defines *how* Suraksha Shadow's functional requirements are implemented – architecture, technology choices, data model, and non-functional requirements.

2. System Architecture (high level)

- **Client (PWA):** React + Vite front end, installable as a Progressive Web App, running on the Primary User's phone
- **Backend API:** Express / FastAPI service handling trigger events, SOS dispatch, and Sahara's RAG pipeline

- **Data & Realtime layer:** Supabase (Postgres + pgvector + Realtime channels)
- **AI layer:** Gemini / Claude API for fake-call dialogue generation and Sahara's response generation
- **Alerting:** Fast2SMS for SMS-based SOS dispatch to Trusted Contacts
- **Deployment:** Netlify (frontend), Render (backend)

Flow: Client detects trigger locally → calls Backend API → Backend writes an emergency event to Supabase and dispatches SOS via Fast2SMS → Supabase Realtime pushes live updates (location, battery, movement, timeline) to a Guardian web view opened by Trusted Contacts → Sahara's chat, once active, queries Supabase pgvector for relevant knowledge-base content before calling the AI API.

3. Technology Stack & Justification

Layer	Choice	Why
Frontend	React / Vite	Fast to build, installable as a PWA, no app-store dependency

Layer	Choice	Why
Backend	Express / FastAPI	Lightweight, quick to stand up on a hackathon timeline
Database	Supabase (Postgres)	Managed Postgres with built-in auth, storage, and Realtime
Vector search	Supabase pgvector	Keeps Sahara's knowledge base in the same database, no separate vector service
Realtime sync	Supabase Realtime	Powers Guardian Mode's live dashboard without a custom WebSocket server
AI reasoning	Gemini / Claude API	Handles both fake-call dialogue and Sahara's grounded responses
Silent detection	Web Speech API + Device Motion API	Free, browser-native, no extra SDKs needed

Layer	Choice	Why
Alerts	Fast2SMS	Simple SMS API, already used and proven in a prior project
Hosting	Netlify (frontend) / Render (backend)	Fast, free-tier friendly, already familiar

4. Data Model (key entities)

Entity	Key Fields
User	id, name, phone, trusted_contacts[], consent_ambient_audio (bool)
TrustedContact	id, user_id, name, phone, relationship
EmergencyEvent	id, user_id, trigger_type, start_time, end_time, status
TimelineEntry	id, emergency_event_id, timestamp, event_type, details

Entity	Key Fields
LocationPing	id, emergency_event_id, lat, lng, battery_pct, movement_status, timestamp
KnowledgeDocument	id, source (BNS / POSH / NGO directory), content, embedding

5. Technical Requirements

5.1 Shield

ID	Requirement
TR-1	Implement keyword-spotting using the Web Speech API, running continuously during an active call or foreground session
TR-2	Implement motion-anomaly detection using the Device Motion API with a lightweight threshold/classifier (no server round-trip needed for detection itself)
TR-3	On trigger, call the backend's <code>/sos</code> endpoint, which writes an <code>EmergencyEvent</code> and dispatches SMS via Fast2SMS within a target of 3 seconds

ID	Requirement
TR-4	Generate fake-call dialogue via a streaming call to the Gemini/Claude API, converted to speech via a text-to-speech service

5.2 Guardian Mode

ID	Requirement
TR-5	Stream location, battery, and movement status to a <code>LocationPing</code> table at a fixed interval (target: every 5–10 seconds) during an active emergency
TR-6	Expose a Supabase Realtime channel scoped to the <code>EmergencyEvent</code> so a Trusted Contact's Guardian web view updates live without polling
TR-7	Gate ambient-audio streaming server-side on the <code>consent_ambient_audio</code> flag; reject the stream if consent is not set
TR-8	Write every state change (trigger, contact notified, location update, resolution) to <code>TimelineEntry</code> for the Guardian view's timeline

5.3 Sahara

ID	Requirement
TR-9	Ingest and embed source documents (BNS sections, POSH Act, NGO directory) into <code>KnowledgeDocument</code> via Supabase pgvector
TR-10	On a user query, embed the query, run a similarity search against <code>KnowledgeDocument</code> , and pass top-k results as context to the Gemini/Claude API
TR-11	Constrain generation to the retrieved context to reduce ungrounded answers on legal/procedural questions

6. Non-Functional Requirements

- **Performance:** SOS dispatch (trigger → SMS sent) within 3 seconds under normal network conditions.
- **Privacy & Consent:** Ambient audio is opt-in only, revocable at any time, and never enabled by default. Location and audio data are retained only for the duration of an emergency plus a short grace period, then deleted.
- **Security:** All data in transit uses TLS; Supabase row-level security restricts each user's data to themselves and their designated Trusted Contacts.

- **Reliability:** SOS dispatch retries on failure (e.g., SMS gateway timeout); the client falls back to cached last-known location if a live GPS fix isn't available.
- **Scalability:** Shield and Sahara are decoupled services, so either can be reused independently in future builds without pulling in the other's dependencies.

7. Assumptions & Constraints

- Requires the Primary User's phone to have microphone, location, and motion-sensor permissions granted.
- Web Speech API browser support varies (best on Chrome-based browsers) – a known constraint for the hackathon demo.
- Requires active internet connectivity; no offline mode in Phase 1.
- BNS and POSH Act content must be manually curated and verified before ingestion, not scraped automatically.